Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 23.12.2025.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaB_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Quick Sort*

Napisati ***quick*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomNumberGen*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – Brisanje neparnih brojeva iz steka, proizvod preostalih

Implementirati metodu koja prima niz celih brojeva. Metoda treba da koristi strukturu podataka steka (klasa **Stack**) za organizaciju i manipulaciju elementima, i treba da izvrši sledeće korake:

1. **Uklanjanje neparnih brojeva**: Prolazom kroz stek, ukloniti sve brojeve koji neparni
2. **Računanje proizvoda preostalih brojeva**: Izračunati proizvod svih brojeva koji su ostali u steku nakon uklanjanja neparnih brojeva.
3. **Vraćanje rezultata**: Kao rezultat, metoda treba da vrati izračunatu proizvod preostalih brojeva u steku.

<u>Dodatni zahtevi za rešavanje zadatka:</u>

- Zadatak rešavati pomoću strukture ***steka*** (klasa ***Stack***) koja je data u okviru šablona zadatka.
- Zadatak rešavati u okviru metode ***public int RemoveOddAndReturnProduct (int[] array)***
- Nakon što se uklone neparni brojevi iz steka i izračuna proizvod preostalih brojeva, stek treba da ostane u istom stanju kao pre početnog procesa. To znači da preostali brojevi moraju biti vraćeni u stek u originalnom redosledu, a svi izmenjeni elementi (oni koji su uklonjeni) ne smeju se vratiti.
- Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

<u>Primer:</u>

- Input: [10, 15, 7, 30, 2, 8]
- Output: Proizvod preostalih brojeva: 4800
  Preostali brojevi u steku: 10, 30, 2, 8
- Input: [5, 2, 8, 3, 4, 6]
- Output: Proizvod preostalih brojeva: 384
  Preostali brojevi u steku: 2, 8, 4, 6

Zadatak 3 – *Josephus Problem*

Klasican **"Josephus problem"** je matematička zagonetka i algoritamska vežba koja potiče iz antičke legende. Imate **n** elemenata (brojeva) raspoređenih u **krug** (cirkularnu listu). Eliminacija se vrši tako što se počne od određenog mesta i broji se **M** elemenata u krugu, pri čemu se **M-ti element eliminiše** iz kruga. Brojanje se nastavlja od sledećeg elementa posle eliminisanog. Proces se ponavlja dok ne ostane **samo jedan element** - **"preživeli"**.

Problem**:**
U klasičnom problemu, **M** je konstantno (npr. svaki 3-či se eliminiše). U našem slučaju, **M se menja posle svake eliminacije** prema formuli:

$$M_{novo} = (M_{staro}\ x\ 2)\ \%\ N_{preziveli} + 1$$

Dodatni zahtevi za rešavanje zadatka:

1. Zadatak rešavati u metodi ***public int JosephusProblem(int startM)*** koja se nalazi u okviru klase ***CircularList***.
2. Parametar ***int startM*** koji se prosleđuje metodi ***JosephusProblem*** zadaje korisnik prilikom prvog poziva ove metode.
3. Svaki čvor liste treba da predstavlja jednog preživelog sa svojim ID brojem (od 1 do n).
4. Obezbediti kod u Main metodi za testiranje.

Primer:

Input (cirkularna lista): 1 -> 2 -> 3 -> 4 -> 5 -> 6

startM: 2

Korak 1: 1 -> 3 -> 4 -> 5 -> 6

Brojanje startuje od: 3 novoM: 5

Korak 2: 3 -> 4 -> 5 -> 6

Brojanje startuje od: 3 novoM: 3

Korak 3: 3 -> 4 -> 6

Brojanje startuje od: 6 novoM: 1

Korak 4: 3 -> 4

Brojanje startuje od: 3 novoM: 1

Korak 5 (preživeli): 4